frontend notes

<- the screen side of full-stack ->

2026 edition - simple words, real tools

1) WHAT IS THIS?

every logo here is a real tool people use

a notebook about the part of an app you can see and touch - the screen

one example all the way: USER orders 2 milk + bread on the app!



HTML
↓
the bones: what is on the page



CSS
↓
the clothes: colour, size, layout



JavaScript
↓
the muscles: taps & changes



TypeScript
↓
JS + types, catches mistakes



React
↓
builds the screen from components



Next.js
↓
React + pages + a server



Tailwind
↓
style with tiny class names



Vite
↓
dev server + build, very fast



TanStack Query
↓
fetch + cache backend data



Figma
↓
the design before the code



DevTools
↓
look inside any page



Vercel
↓
puts the site on the internet

2) ONE IDEA PER TILE

logo = the real tool, small words = what it does for USER

frontend = the shop window. it shows things, takes USER's taps, and asks the backend for everything!

3) THE BIG PICTURE

a website is like a shop window: HTML is the shelves, CSS the paint and lights, and JavaScript the shopkeeper who reacts when you knock

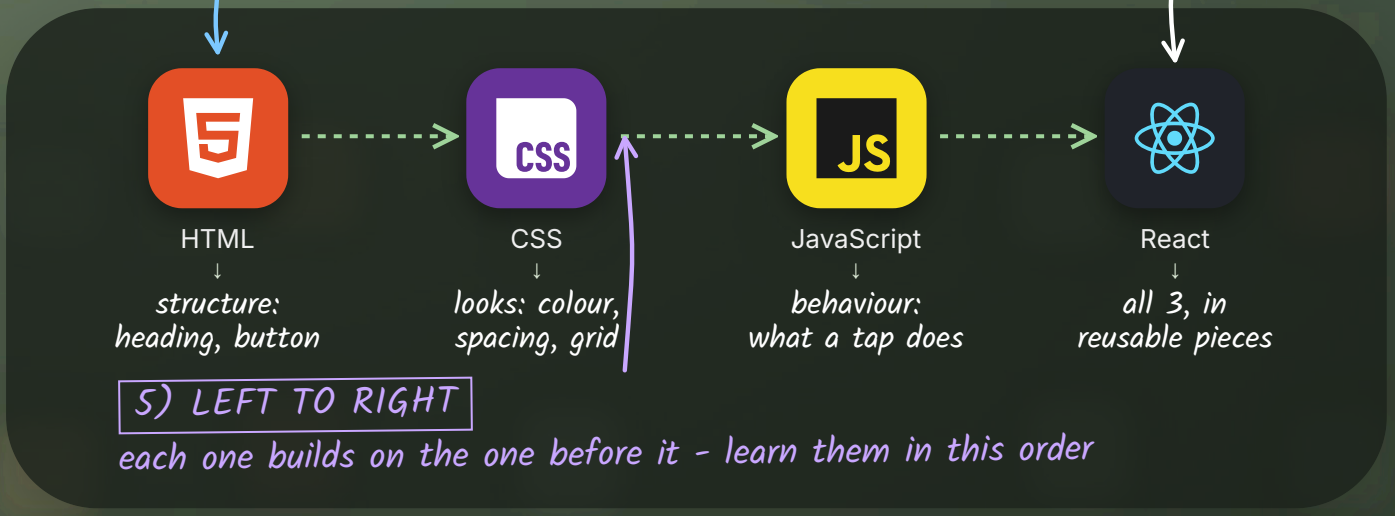
introduction

<- what these notes are about ->

4) THE BODY IDEA

a web page is like a body: HTML = bones, CSS = clothes, JavaScript = muscles. React is a tidy way to build all three in small pieces

the 4 big pieces (remember these!)



5) LEFT TO RIGHT

each one builds on the one before it - learn them in this order

FRONTEND

everything USER sees and touches: the page, the buttons, the forms. it runs INSIDE the browser on USER's phone or laptop, not on our server. built with HTML + CSS + JavaScript, today mostly through React (often with TypeScript + Next.js).

6) FRONTEND IN ONE LINE

everything USER sees and taps, running on USER's own phone

HOW IT TALKS TO THE BACKEND

the frontend never keeps the real data. it asks the backend with fetch(): GET /api/products, POST /api/orders. the backend answers in JSON and React turns that JSON into cards and buttons on the screen.

7) TALKING IN ONE LINE

the screen sends a note, the server writes back, the screen shows the answer

WHAT GOOD LOOKS LIKE

fast (something on screen in under 1 s), works on phone + laptop, clear loading and error screens, usable by everyone (keyboard, screen reader, good contrast), and never trusts itself - the server checks everything again.

8) EASY WAY TO REMEMBER

HTML says WHAT is there, CSS says how it LOOKS, JavaScript says what it DOES, React keeps it all tidy - and the backend keeps the real data

one tap

<- what happens when USER taps ADD, start to end ->

all of this in ~16 ms
= one screen frame!

9) WHAT IS A TAP?

for the app, a tap is an 'event' - a tiny message that says 'someone pressed me'

no backend call yet -
the cart lives in the browser



10) ONLY WHAT CHANGED

React repaints just the one button - like fixing one tile, not the whole floor

state changes -> React re-renders -> browser paints.
that loop is the heart of every React app

tip: never change the page by hand (DOM).
change the STATE and let React redraw.

11) LIKE A LIGHT SWITCH

you press, the room changes, you never see the wires. here the 'wires' are 8 quick steps, all finished before your finger leaves the glass

components

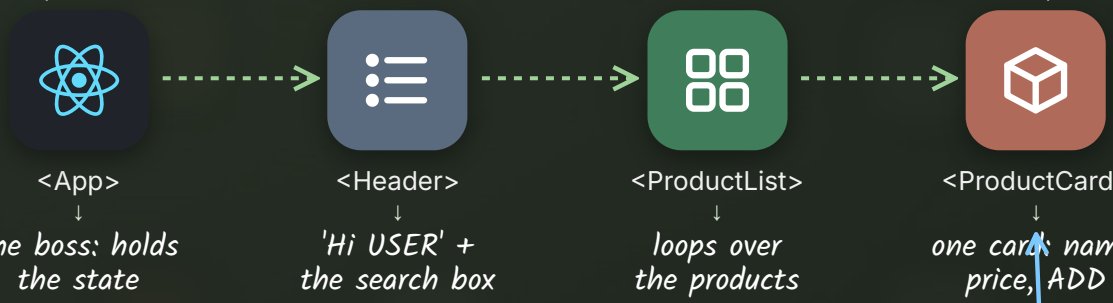
<- small pieces that snap together ->

12) WHAT IS A COMPONENT?

one small piece of the screen with its own job - a header, a card, a button bar

a component = a function that returns UI

same <ProductCard> used 6 times!

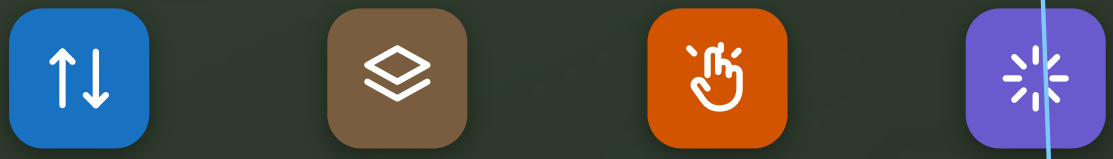


<App>
↓
the boss: holds the state

<Header>
↓
'Hi USER' + the search box

<ProductList>
↓
loops over the products

<ProductCard>
↓
one card: name, price, ADD



Props
↓
data going DOWN: product, qty

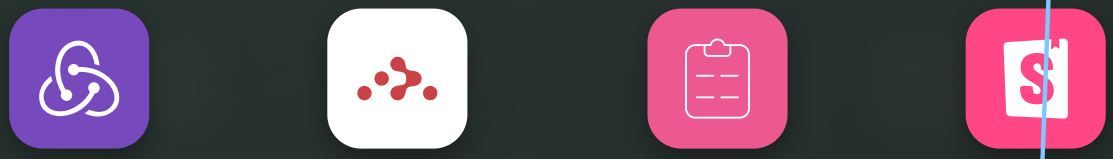
State
↓
data that CHANGES: cart, search text

Events
↓
taps going UP: onQty(id, +1)

Effect
↓
run after render: load products once

the rules

helpers



Shared store
↓
Redux / Zustand: state many pages need

Router
↓
/ -> /checkout without reloading

Forms
↓
values + errors without pain

Storybook
↓
build & check each card alone

13) BUILD ONCE

the same card shape is reused for every product

props go down, events come up.
keep each piece of state in ONE place
(the closest parent that needs it)

14) LIKE LEGO

small bricks build big things, and one brick can be used many times. props = what a parent tells a child, state = what a part remembers by itself

talking to the backend

<- getting data in and out of the screen ->

the frontend NEVER owns the real data

15) THE FRONT ASKS

the screen has no real data of its own - it asks the server

CORS = the browser asks: may this site call that API?



fetch()

↓

GET /api/products -> JSON list of products



Token

↓

log in once, send 'Bearer ...' with every request



Loading

↓

grey skeleton cards, never a blank screen



Error

↓

'Couldn't load' + a Try again button



TanStack Query

↓

fetch + cache + retry, done for you



Cache

↓

products saved: go back = instant



Live updates

↓

SSE / WebSocket: 'rider is 2 min away'



Optimistic UI

↓

show 'added' now, undo if the server says no

16) KEEP A COPY

show the saved copy at once, swap in fresh data when it arrives

every screen has 4 states: loading, error, empty, ready.
design all 4, not only the happy one!

the token lives in memory (or an httpOnly cookie)
- never put secrets or API keys in frontend code

17) LIKE ORDERING BY PHONE

you call and wait (loading), sometimes the line drops (error), you call again (retry). a good app always tells USER which of these is happening, so nobody stares at a blank screen

looks good, works for all

<- style it, fit every screen, let everyone in ->

Figma design first, then CSS

18) THREE JOBS

a page should look good, fit any screen and work for everyone

1 in 6 people has some disability - build for them

styling

CSS
↓
colour, spacing, fonts, shadows

Tailwind
↓
'p-4 rounded-xl' right in the JSX

Flex & Grid
↓
rows + columns: the 2-wide product grid

shadcn/ui
↓
ready buttons & dialogs you own

any screen

Phone first
↓
design 390 px wide first

Media query
↓
900 px+ -> 4 columns + side cart

Dark mode
↓
same app, dark colours

Motion
↓
small animations that explain

for everyone

Keyboard
↓
Tab reaches every button, ring shows where

Contrast
↓
text vs background at least 4.5 : 1

Screen reader
↓
aria-label='Add one' is read aloud

Lighthouse
↓
scores speed + access, 0-100

19) ALL THREE ROWS

pretty colours alone are not enough

accessibility (a11y) is not extra work at the end - real buttons, labels and contrast from day one

20) WATER IN A GLASS

responsive = water takes the shape of any glass. accessibility = a ramp next to the stairs: built for some, used by everyone (prams, suitcases, tired legs)

USER's order

<- the same order, seen from the browser ->

link to 'placed!' - 12 steps

21) ONE STORY

the same order as in the backend notes, now seen from USER's phone

the backend notes show the server half of this same trip



load (1-7) + tap (8) + form (9-10) + server (11) + result (12) = one smooth screen for USER

22) LOAD, LISTEN, TALK

first the app LOADS (steps 1-7), then it LISTENS for taps (8), checks the form (9-10) and TALKS to the server (11) before showing the result (12)

2026 toolbox

<- what people actually use right now ->

learn the IDEA, not the logo

23) WHY SO MANY?

each tool fixes one pain - looks, data, forms or tests

Svelte, Vue, Astro are great too - same ideas



TypeScript
↓
JS with types



React
↓
UI in components



Next.js
↓
full-stack React



Vite
↓
dev + build



Tailwind
↓
utility CSS



shadcn/ui
↓
copy-in components



Radix
↓
accessible parts



TanStack Query
↓
server data



Redux / Zustand
↓
app-wide state



React Router
↓
pages / URLs



React Hook Form
↓
forms



Zod
↓
check the data



Vitest
↓
unit tests



Storybook
↓
component lab



Figma
↓
design + handoff



Vercel
↓
deploy in 1 click

your next step: give every project a loading + error screen and a Lighthouse score -> big portfolio boost

24) PENS, NOT WRITING

tools are like brands of pens - any pen writes. start with HTML + CSS + JS, then React, then one framework. pick up the rest when you need it

real screens

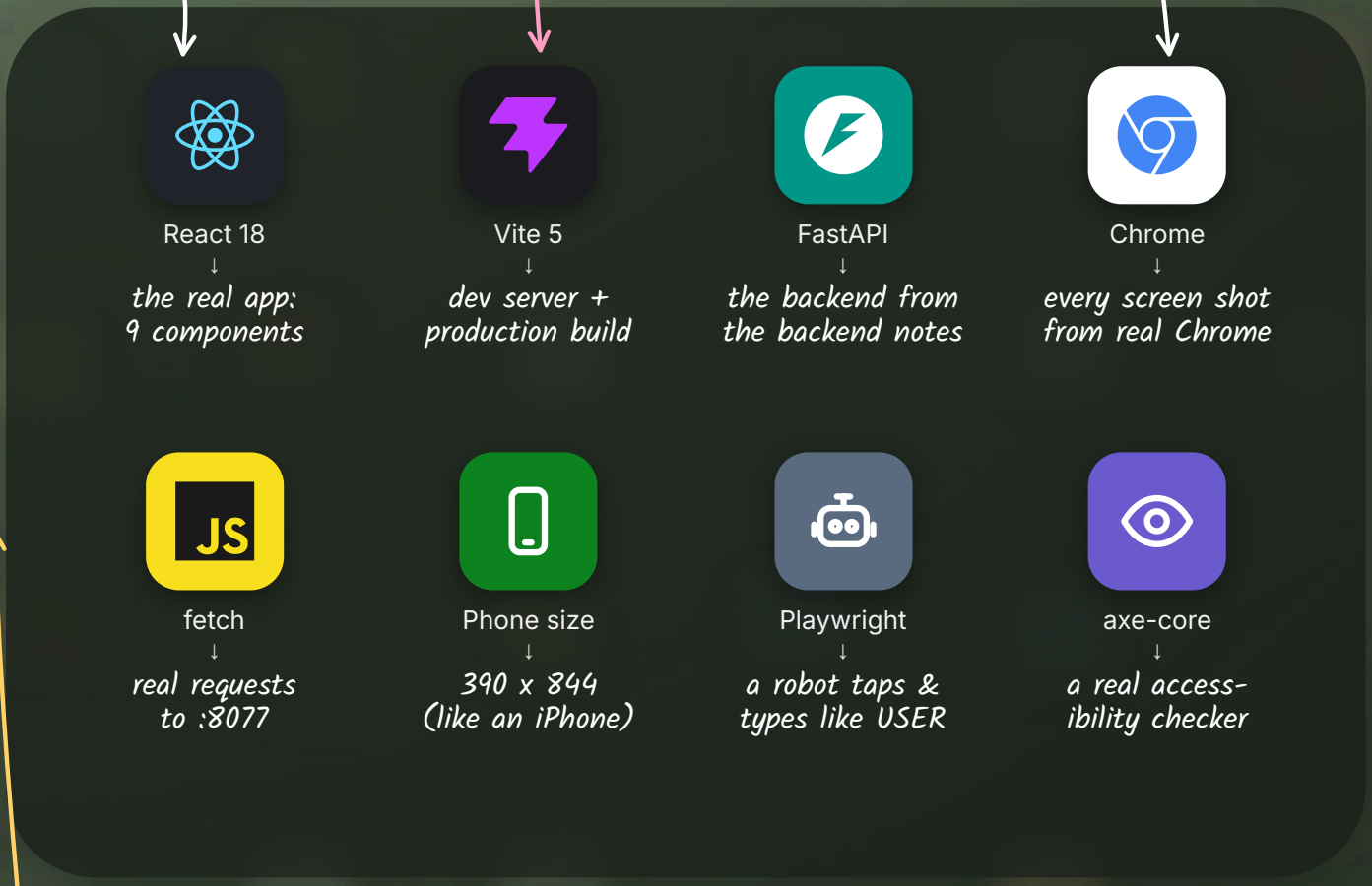
<-- part 2: the same notes, really running -->

25) MAP VS ROAD

part 1 was the map. part 2 is the real road: a real React app, really running

I built a small Quick Grocery app in React

every screenshot is from 9 Nov 2026



100% real: the React code, the requests, all 4 screen states, form checks, the saved order, bundle size, timings, ally scan.

said honestly: the loading screen was held for a moment on purpose so you can see it; the 'no internet' was faked by the robot.

run it yourself: cd projects/frontend-notes-pdf
./run.sh (starts backend + React, captures, builds this PDF)

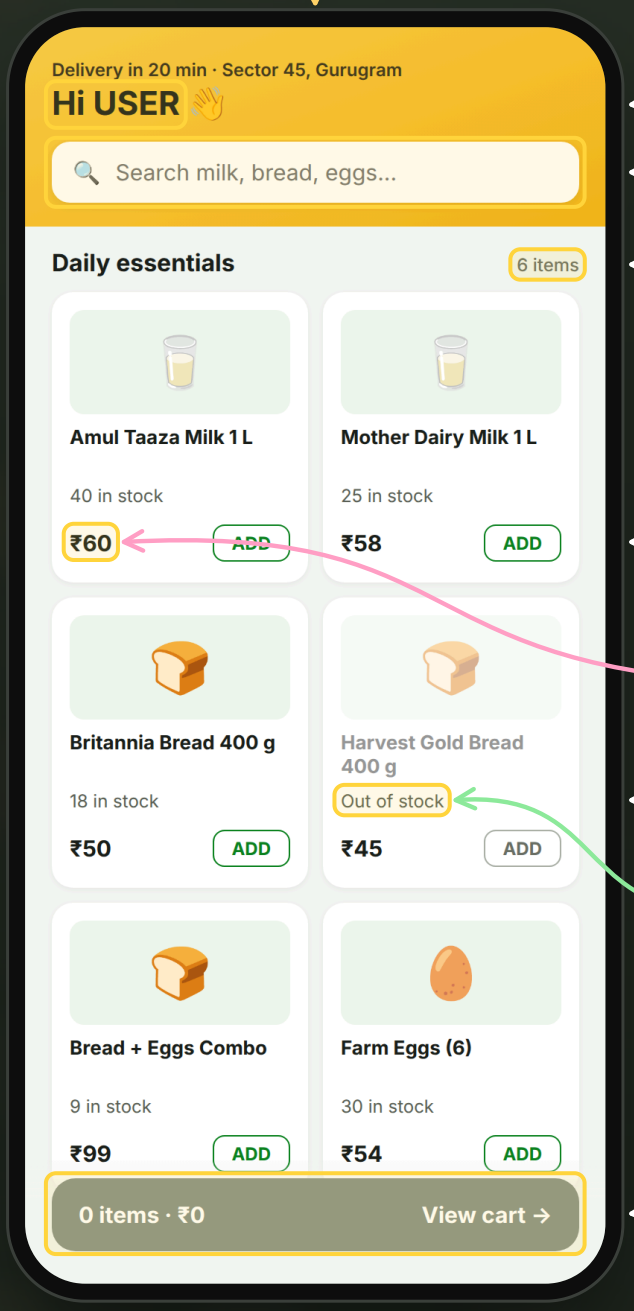
26) WHY REAL SCREENS?

a robot tapped the app just like USER would and took these pictures. nothing is drawn by hand except these notes

the app

<- what USER sees on the phone ->

a phone screen is small, so the most important things (the products and the cart button) get the most space



<Header name='USER' />
-> 'Hi USER'

search box = state 'query'.
type -> list filters live

'6 items' is counted from the data, not typed in

Rs 60 came from
GET /api/products (JSON)

27) FRESH PRICES

asked from the server every time

stock 0 -> 'Out of stock'
+ ADD button disabled

28) GREY = NOT NOW

a faded card says 'can't buy' without words

<CartBar>: grey until USER adds something

the screen is only a picture of the DATA.
change the data -> React redraws the picture.

29) THUMB ZONE

the big green button sits at the bottom, right where USER's thumb already is. good design puts the main action where the hand rests

one component, for real

<- ProductCard.jsx - the code behind each card ->

a component is a recipe card for one part of the screen:
give it ingredients (props), it gives back the finished part

src/components/ProductCard.jsx - VS Code

```jsx
export default function ProductCard({ product, qty, onQty }) {
  const soldOut = product.stock === 0
  return (
    <article className={soldOut ? 'card soldout' : 'card'}>
      <div className="img" role="img" aria-label={product.name}>{emojiFor(product.name)}</div>
      <b>{product.name}</b>
      <span>{soldOut ? 'Out of stock' : `${product.stock} in stock`}</span>
      <div className="row">
        <strong>{`₹${product.price}`}</strong>
        {qty === 0 ? (
          <button className="add" disabled={soldOut} onClick={() => onQty(product.id, +1)}>ADD</button>
        ) : (
          <div className="stepper">
            <button aria-label="Remove one" onClick={() => onQty(product.id, -1)}>−</button>
            <output>{qty}</output>
            <button aria-label="Add one" onClick={() => onQty(product.id, +1)}>+</button>
          </div>
        )}
      </div>
    </article>
  )
}
```

props in: which product, how many, what to call

a rule in JS: stock 0 = sold out

JSX = HTML-looking code inside JavaScript

tap -> tell the parent (event goes UP)

30) WHY SPLIT INTO PIECES?

one card file, used 6 times. fix a mistake once and every card is fixed

25 lines. used 6 times on the screen, once per product.

31) LOOKS + LOGIC TOGETHER

JSX looks like HTML but lives inside JavaScript, so how a card looks and what it does sit side by side

state in action

<- USER taps ADD, +, ADD (page 3, for real) ->

state is the app's short-term memory: what's in the cart right now, what you typed in search. change it and the screen updates itself

cart = {11: 2, 21: 1}
that one object is the whole truth

milk: ADD became [- 2 +]
= qty prop is now 2

32) THE ONE TRUTH

the whole cart is one small list. everything on the screen is drawn from it

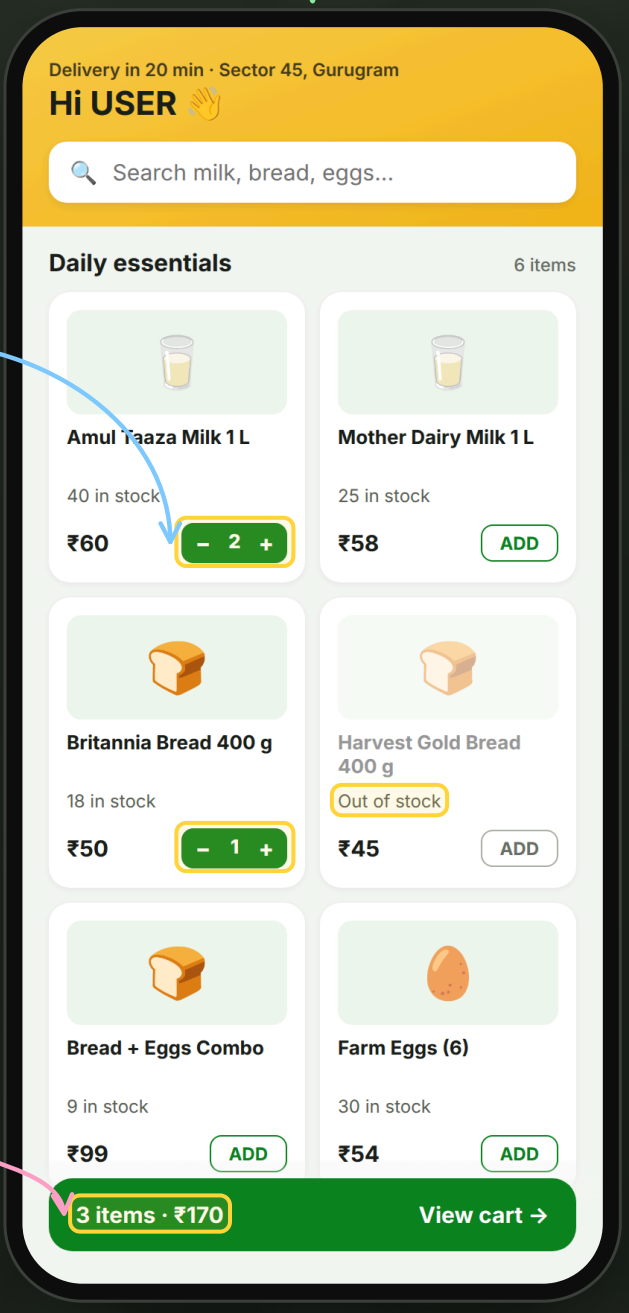
bread: qty 1

sold out stays sold out
(nothing re-drawn here)

33) MATHS ON THE SCREEN

the total isn't saved anywhere - it is worked out again from the cart every time

CartBar did the maths:
2 x 60 + 1 x 50 = Rs 170



no server call yet - the cart lives in React state

34) SHORT-TERM MEMORY

state is the app's short-term memory. close the tab and it's gone - that's why real orders are saved on the server

where the state lives

<- read live out of React while the app ran ->

keep each memory in ONE place, the closest parent that needs it - like keeping the family's house keys on one hook

React component tree (read live from the running app)

```
<App>   6 state
  <Header> name query onQuery
  <ProductList> status products query cart onQty onRetry
    <ProductCard> product qty onQty  "Amul Taaza Milk 1 L"  qty=1
    <ProductCard> product qty onQty  "Mother Dairy Milk 1 L"  qty=0
    <ProductCard> product qty onQty  "Britannia Bread 400 g"  qty=0
    <ProductCard> product qty onQty  "Harvest Gold Bread 400 g"  qty=0
    <ProductCard> product qty onQty  "Bread + Eggs Combo"  qty=0
    <ProductCard> product qty onQty  "Farm Eggs (6)"  qty=0
  <CartBar> lines total onCheckout
```

App holds 6 pieces of state

qty=1 flows DOWN as a prop

src/App.jsx - VS Code

```jsx
export default function App() {
  const [products, setProducts] = useState([])
  const [status, setStatus] = useState('loading') // loading | ready | error
  const [cart, setCart] = useState({})            // { productId: qty }
  const [query, setQuery] = useState('')
  const [screen, setScreen] = useState('home')    // home | checkout | done
  const [order, setOrder] = useState(null)

  async function load() {
    setStatus('loading')
    try {
      await login('user@example.com')
      setProducts(await getProducts())
      setStatus('ready')
    } catch (e) {
      setStatus('error')
    }
  }
  useEffect(() => { load() }, [])

  function changeQty(id, delta) {
    setCart(c => {
      const qty = (c[id] || 0) + delta
      const next = { ...c, [id]: qty }
      if (qty <= 0) delete next[id]
      return next
    })
  }
```

useState = a box React watches. setCart -> redraw

useEffect: when the app opens, load once

talking to the backend

<- DevTools > Network, while the app opened ->

every time the app needs something it sends a request -
like posting a letter and waiting for the reply

Chrome DevTools - Network (recorded while the app loaded)

Name	Method	Status	Type	Size	Time
/	GET	200	document	0.4 kB	6 ms
/assets/index-ChgUmVS0.css	GET	200	stylesheet	4.8 kB	11 ms
/assets/index-D93B9Ng3.js	GET	200	script	145.9 kB	32 ms
/api/login	POST	200	fetch	0.2 kB	24 ms
/api/products	GET	200	fetch	0.7 kB	7 ms

5 requests · 152 kB

login first -> token

then the product list
(0.7 kB of JSON)

35) WRISTBAND

the token is like a wristband at a concert: show it at every door and you're let in

src/api.js - VS Code

```
1  // every call to the backend goes through here
2  let token = null
3
4  async function call(path, options = {}) {
5    const res = await fetch(path, {
6      ...options,
7      headers: {
8        'Content-Type': 'application/json',
9        ...(token ? { Authorization: 'Bearer ' + token } : {}),
10     },
11   })
12   const data = await res.json().catch(() => ({}))
13   if (!res.ok) throw new Error(data.detail?.[0]?.msg || data.detail || 'HTTP ' + res.status)
14   return data
15 }
16
17 export async function login(email) {
18   token = (await call('/api/login', { method: 'POST', body: JSON.stringify({ email }) })).token
19   window.TOKEN = token
```

one helper for every call:
fetch + JSON

token rides along in
the Authorization header

36) ONE HELPER

every server call goes through one small
function - fix it once, every screen benefits

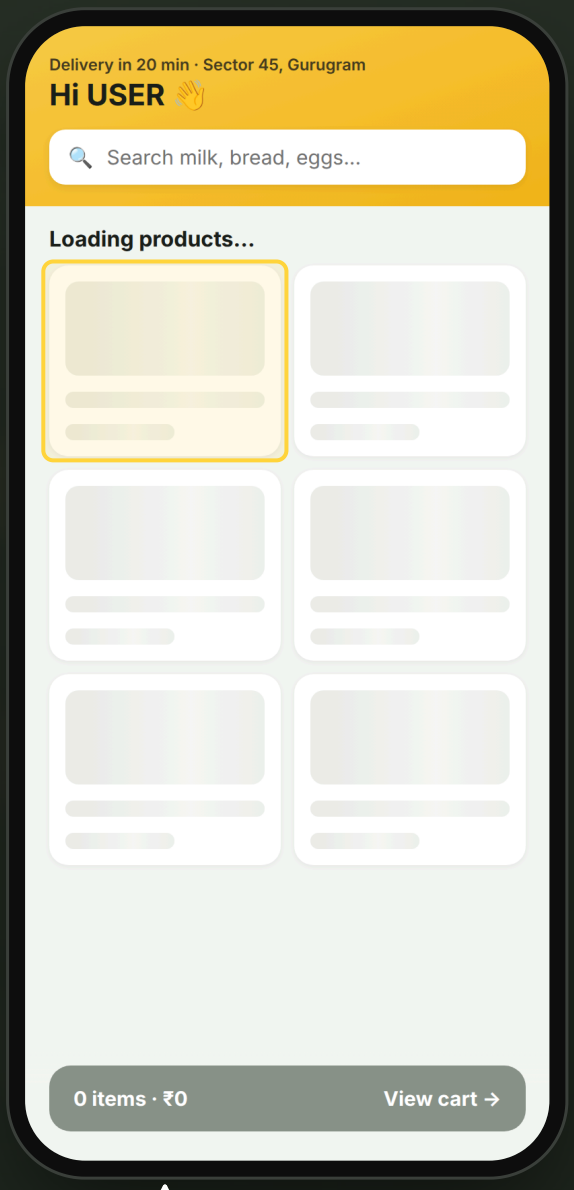
server said no? -> throw,
the screen shows an error

loading + errors

<- the screens nobody designs (but should) ->

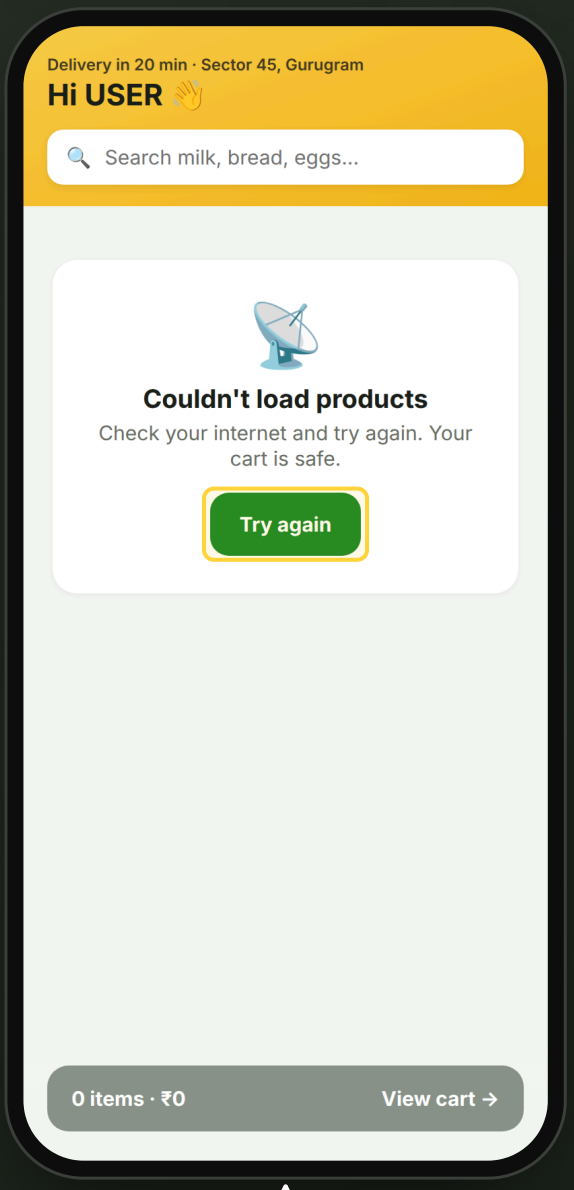
waiting and failing are normal on the internet -
a good app always tells you what is happening

LOADING (held 2.5 s)



skeleton = grey shapes,
so it never looks broken

ERROR (internet cut)



kind words + what to do
+ Try again (it worked!)

header still shows -> USER knows the app is alive

37) SKELETON

like a table already set before the food arrives - USER knows something is coming

form checks

<- wrong data is stopped BEFORE it is sent ->

checking a form = a shop asking 'is this the right number?' before packing your order - quick, polite, saves a wasted trip

the cart from page 12 = Rs 170

38) WHY HERE?

catching a typo on the phone is instant - no trip to the server

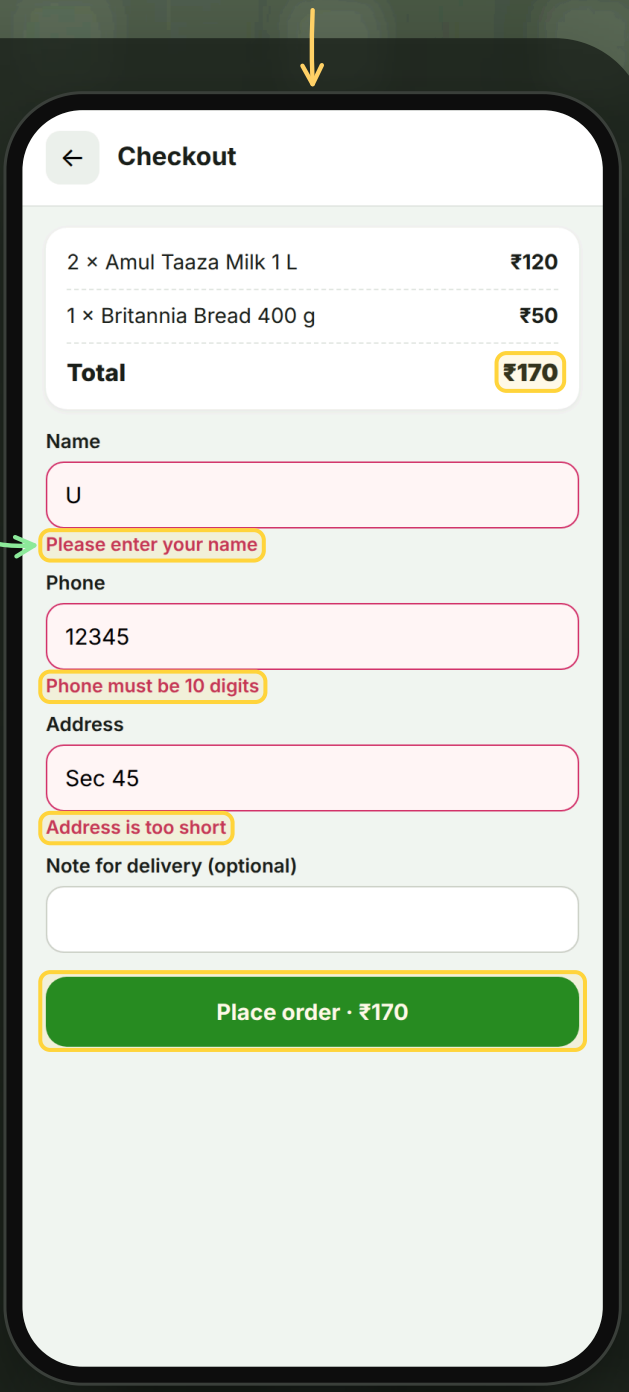
'U' -> too short

'12345' -> phone must be 10 digits

'Sec 45' -> too short

tapped, but NO request went out - fix first

red border + message under the box + role='alert' for screen readers



frontend checks = fast + friendly. backend checks = safe. do BOTH.

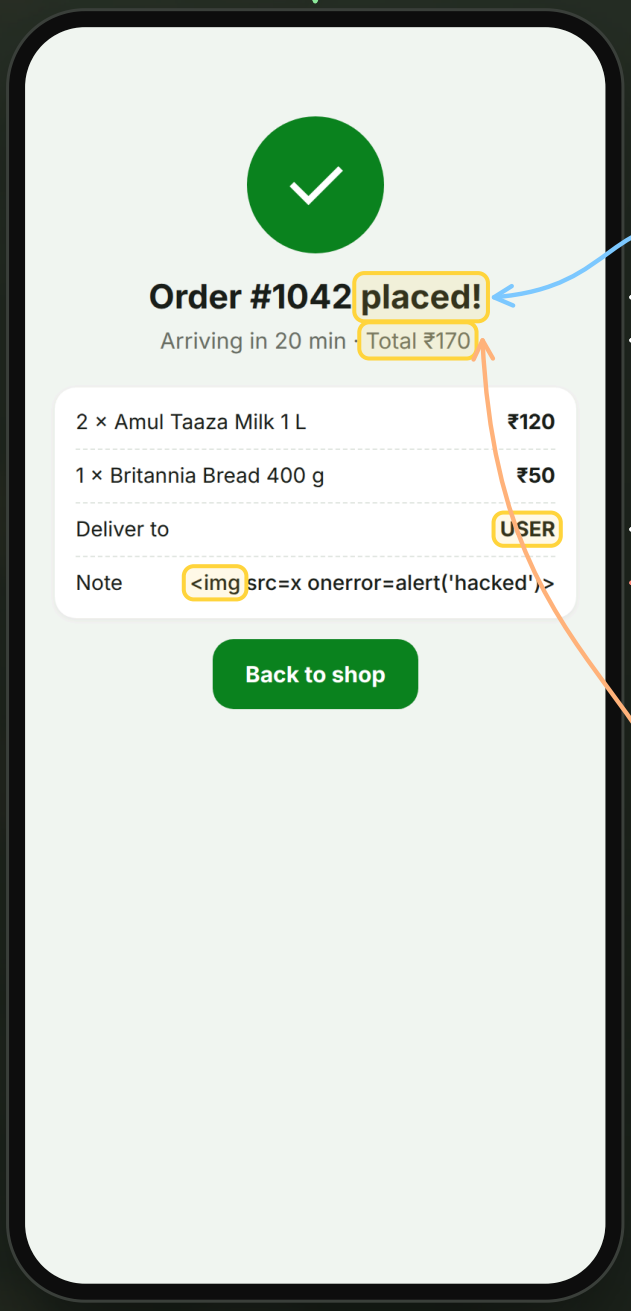
39) CHECK TWICE

the server checks AGAIN later - a clever person can skip the phone's checks, never the server's

order placed!

<- right data -> real order ->

a full round trip: the form went to the server, the server saved it and sent back the order number you see here



40) ROUND TRIP

form -> server -> database -> answer -> screen, all in under a second

#1042 = a real row in the backend database

total from the server's answer, not the phone

name from the form

a sneaky note: an HTML tag that runs code... (next page!)

41) TRUST

every number here came from the server

form -> fetch -> FastAPI -> database -> JSON -> React draws 'placed!'

42) THE TICK

the big green tick is a small reward: it tells USER 'done, you can relax'. little happy moments like this make people trust an app

the request + safety

<- what Place order really sent ->

only small facts travel (which product, how many). the server looks up prices itself, so nobody can change them on their phone

Chrome DevTools - the order request (recorded)

```
POST /api/orders   ->   200 OK

Request headers
  content-type: application/json
  authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey...

Request body (sent by placeOrder)
  {"items":[{"product_id":11,"qty":2},{"product_id":21,"qty":1}]}

Response (from FastAPI)
  {"status": "placed", "order" "#1042", "total": 170, "items": [{"name": "Amul Taaza Milk 1 L", "qt
y": 2, "price": 60}, {"name": "Britannia Bread 400 g", "qty": 1, "price": 50}]}
```

only ids + qty go up. the SERVER sets prices

200 OK = saved, here is the order it made

browser console - is the note dangerous?

```
# a classic XSS test: an <img> tag that runs code
user-demo $ the note USER typed
<img src=x onerror=alert('hacked')>

user-demo $ React shows it with {order.note}
alerts that fired: 0   (shown as plain text)

# the unsafe way
user-demo $ same text with el.innerHTML = note
alerts that fired: 1   -> 'hacked'
```

React: 0 alerts, the tag was shown as plain text

innerHTML: 1 alert fired - the code RAN (= XSS)

43) XSS

a stranger hiding a trick inside normal-looking text. React treats typed text as plain words, never as orders

React escapes text for you. avoid innerHTML / dangerouslySetInnerHTML with anything a user typed.

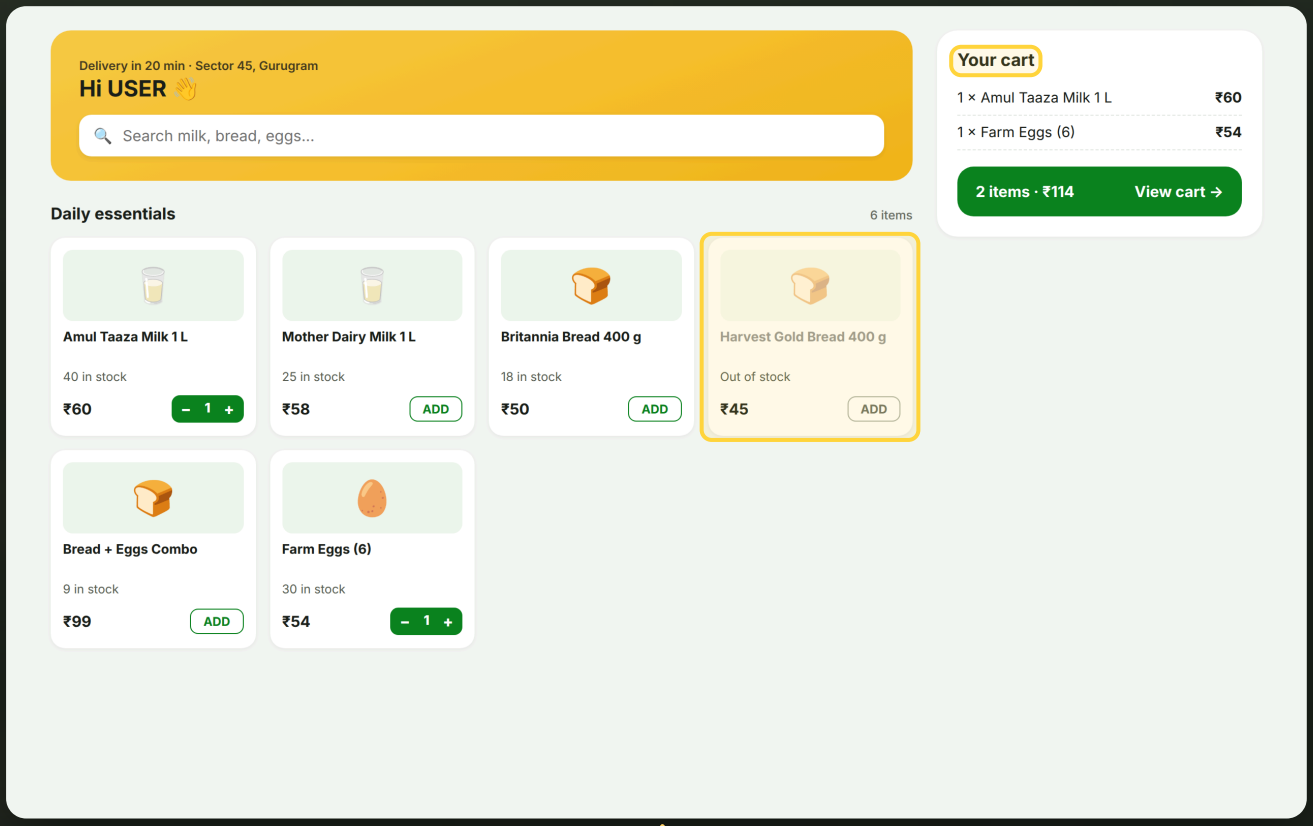
44) ONLY SMALL FACTS

the phone sends just 'which product, how many'. the server looks up prices itself, so nobody can change them on their phone

phone + laptop

<- same code, a 1280 px wide laptop screen ->

responsive design = water in a glass: the same app takes the shape of any screen, from a phone to a big laptop



4 columns now (2 on the phone)

45) SAME CODE

one app, every screen size

cart moved to the side, sticky while you scroll

src/styles.css - VS Code

```
/* bigger screens: 4 columns + cart on the side */
@media (min-width: 900px) {
  .layout { display: grid; grid-template-columns: 1fr 320px; max-width: 1240px; margin: 0 auto; gap: 24px; padding: 24px }
  .top { border-radius: 20px; padding: 24px 28px }
  .grid { grid-template-columns: repeat(4, 1fr); gap: 14px; padding: 8px 0 30px }
  .sec { padding: 22px 0 6px }
  .cartbar { position: sticky; top: 24px; align-self: start; background: #fff; border-radius: 20px; padding: 20px; box-shadow: 0 1px 4px rgba(0,0,0,.06) }
  .cartbar .side { display: block } .cartbar h2 { margin: 0 0 10px; font-size: 17px }
  .cartbar ul { list-style: none; padding: 0; margin: 0 0 16px }
  .cartbar li { display: flex; justify-content: space-between; gap: 8px; padding: 8px 0; border-bottom: 1px dashed var(--line); font-size: 14px }
  .cartbar li.empty { color: var(--mut) }
}
```

the one rule that does it

works for everyone

<- keyboard + a real accessibility scan ->

accessibility means everyone can use it: people who can't see well, can't use a mouse, or are holding a baby in one arm

axe-core 4.10.2 - accessibility check (WCAG 2 A + AA)

```
# run inside the real page in Chrome
user-demo $ axe.run(document)
rules passed : 20
rules failed : 1
  x color-contrast (serious, 1 place): Elements must meet minimum color contrast ratio thresholds
    where: .soldout > b -> Element has insufficient color contrast of 2.82 (foreground color: #999a9
9, background color: #ffffff, font size: 9.8pt (13px), font weight: bold). Expected contrast ratio of
4.5:1
```

20 rules passed

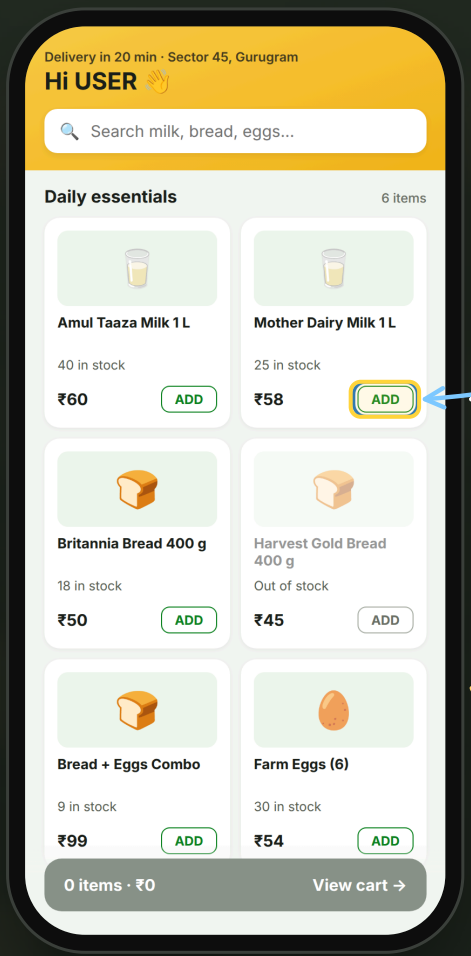
1 real problem found: faded sold-out text is too light (2.82 : 1)

46) WHAT IS A11Y?

short for accessibility: everyone can use the app - weak eyes, no mouse, or just a busy day

47) CONTRAST

light grey on white is hard to read in sunlight - aim for 4.5 : 1



pressed Tab 3 times: the blue ring shows where you are (ADD on Mother Dairy)

48) KEYBOARD

Tab must reach every single button

fix for the problem: fade only the picture, keep the name dark. scanners catch what eyes miss - run one every release.

49) A RAMP FOR ALL

a ramp next to the stairs is built for wheelchairs, but parents with prams and people with suitcases use it too. screen readers read the page out loud, so every button needs a clear name like 'Add one'

speed

<- how heavy + how fast, measured ->

speed = how quickly USER sees something useful.
most people leave an app that takes more than ~3 seconds

vite build - production bundle

```
# turn src/ into small files for the browser
user-demo $ npx vite build
✓ 40 modules transformed.
dist/index.html                   0.40 kB | gzip:  0.27 kB
dist/assets/index-ChgUmVS0.css    4.89 kB | gzip:  1.63 kB
dist/assets/index-D93B9Ng5.js   149.34 kB | gzip: 48.37 kB
✓ built in 641ms
```

all our JS: 149 kB, zipped to 48 kB

React itself is most of that - our code is tiny

50) LIGHT BAG

smaller files = faster app, like a lighter bag is quicker to carry. most of our 48 kB is React itself

Web Vitals - measured in Chrome (local laptop)

```
# read the browser's own timers
user-demo $ performance.getEntries...
TTFB  first byte from server   4 ms
FCP   first thing painted      68 ms
LCP   biggest thing painted    100 ms
CLS   layout jumping           0.000
load  page fully loaded        34 ms
```

LCP 100 ms: the main content showed (good < 2500)

CLS 0: nothing jumped around while loading

51) WHAT THE TIMERS MEAN

LCP = when the main thing appears. CLS = how much things jump around while loading

honest: this is a laptop talking to itself. a real phone on 4G is slower - test with DevTools 'Slow 4G' too.

52) 3 SECONDS

most people leave an app that takes more than ~3 seconds to show something

that's the whole frontend - real, working, on one laptop